

Module 4: Critical Thinking
Option 1: Toxicology Testing

Christine DeLuna
CSC 580: Dr. Nguyen
August 15, 2026

Introduction

The toxicology classification option was selected because of its direct applicability to biomedical research and its relevance to the growing use of artificial intelligence in healthcare. Predicting whether a chemical compound is toxic is an important challenge in pharmaceutical development, environmental health, and clinical research. Traditional toxicity testing often requires extensive laboratory experimentation, making the process both time-consuming and expensive. Neural networks provide an alternative approach by learning complex relationships within large chemical datasets, allowing researchers to identify patterns that may not be readily apparent using traditional statistical methods (Goodfellow et al., 2016).

The objective of this project was to develop and evaluate a feedforward neural network capable of classifying compounds from the Tox21 dataset as either toxic or non-toxic. The model was implemented using TensorFlow and DeepChem, trained using mini-batch gradient descent, and monitored using TensorBoard to evaluate model convergence throughout the training process. In addition to assessing the model's predictive performance, this paper discusses the data preprocessing steps, neural network architecture, training methodology, and the challenges encountered while adapting legacy TensorFlow code to a modern TensorFlow 2.x environment. Although this assignment focuses on toxicology prediction, the workflow demonstrates neural network concepts that are broadly applicable to biomedical research and other healthcare-related machine learning applications.

Dataset and Data Preprocessing

The neural network was trained using the Tox21 dataset, a publicly available benchmark dataset distributed through the DeepChem machine learning framework. The Tox21 dataset was developed as part of the Toxicology in the 21st Century (Tox21) initiative, which seeks to improve toxicity screening by combining high-throughput biological assays with computational modeling. Each compound within the dataset is represented by a 1,024-dimensional molecular fingerprint that captures structural characteristics of the chemical compound and serves as the input features for the neural network (Ramsundar et al., 2019).

The original Tox21 dataset contains multiple toxicity prediction tasks; however, the assignment focused on binary classification using only the first toxicity task. During preprocessing, the feature matrices, labels, and sample weights were extracted from the DeepChem dataset objects. The label and weight arrays were then reduced to a single target variable by selecting the first column of each multidimensional array. This preprocessing step simplified the problem into a binary classification task while maintaining the molecular fingerprint representation required by the neural network. After preprocessing, the dataset was divided into separate training, validation, and testing subsets provided by DeepChem, allowing the model to learn from one set of compounds while evaluating performance on previously unseen data.

Figure 1. *Successful Loading of the Tox21 Dataset Using DeepChem and TensorFlow*

```
TensorFlow Version: 2.21.0
DeepChem Version: 2.8.0

Loading the Tox21 dataset...
Dataset loaded successfully!
Training samples: 6258
Validation samples: 782
Testing samples: 783

Process finished with exit code 0

> assignments > CTA4-Toxicology-Classification > tox21_classifier.py
```

Figure Note. *Successful execution of the DeepChem Tox21 dataset loading process using TensorFlow 2.21.0 and DeepChem 2.8.0. After resolving compatibility issues between RDKit and the scientific Python environment, the dataset loaded successfully, producing 6,258 training samples, 782 validation samples, and 783 testing samples for use in the toxicology classification model.*

Figure 2. *Summary of the Tox21 Dataset Following Preprocessing*

```
Loading the Tox21 dataset...
Dataset loaded successfully!
Training samples: 6258
Validation samples: 782
Testing samples: 783

Task reduction complete.
Training labels: (6258,)
Validation labels: (782,)
Testing labels: (783,)

Dataset Summary
-----
Training Features : (6258, 1024)
Training Labels   : (6258,)
Validation Features : (782, 1024)
Validation Labels  : (782,)
Testing Features  : (783, 1024)
Testing Labels    : (783,)

TensorFlow graph initialized.

Process finished with exit code 0
```

Figure Note. *Console output confirming completion of the Tox21 data preprocessing pipeline. The original multi-task toxicity labels were reduced to a single binary classification task, resulting in 6,258 training samples, 782 validation samples, and 783 testing samples. Each compound is represented by a 1,024-dimensional molecular fingerprint that serves as the input to the neural network. The figure also confirms successful initialization of the TensorFlow computation graph prior to model construction and training.*

Neural Network Architecture

The toxicology classification model was implemented as a feedforward neural network using TensorFlow 2.21 with TensorFlow 1.x compatibility functions. The network accepts molecular fingerprints consisting of 1,024 numerical features as input, corresponding to the structural representation of each chemical compound. Input placeholders were defined to accommodate mini-batches of varying sizes during training, allowing the model to efficiently process groups of compounds rather than the entire dataset simultaneously.

The hidden layer consisted of 50 neurons using the Rectified Linear Unit (ReLU) activation function. ReLU introduces nonlinearity into the network while helping mitigate the vanishing gradient problem commonly associated with deep neural networks (Goodfellow et al., 2016). To reduce overfitting and improve generalization, a dropout layer was applied after the hidden layer. During training, dropout randomly deactivates a subset of neurons, encouraging the network to learn more robust feature representations rather than relying on a small number of highly influential neurons.

The output layer consisted of a single neuron with a sigmoid activation function, producing a probability that each compound belonged to the toxic class. Binary cross-entropy was used as the loss function because the objective was binary classification. Model parameters were optimized using the Adam optimization algorithm, which adapts the learning rate during training to improve convergence and computational efficiency (Kingma & Ba, 2015). Together, these components formed the complete neural network architecture used for toxicity prediction.

Figure 3. *Mini-Batch Training Progress for the Tox21 Neural Network*

```
===== Epoch 1/10 =====
Batch 0 | Samples 0- 100 | Loss = 1119.2439
Batch 10 | Samples 1000-1100 | Loss = 1123.0687
Batch 20 | Samples 2000-2100 | Loss = 961.3331
Batch 30 | Samples 3000-3100 | Loss = 1077.5378
Batch 40 | Samples 4000-4100 | Loss = 1208.4250
Batch 50 | Samples 5000-5100 | Loss = 886.9487
Batch 60 | Samples 6000-6100 | Loss = 755.4257
Epoch 1 complete.

===== Epoch 2/10 =====
Batch 0 | Samples 0- 100 | Loss = 455.6473
Batch 10 | Samples 1000-1100 | Loss = 321.2740
Batch 20 | Samples 2000-2100 | Loss = 414.9475
Batch 30 | Samples 3000-3100 | Loss = 272.1594
Batch 40 | Samples 4000-4100 | Loss = 584.0844
Batch 50 | Samples 5000-5100 | Loss = 440.4092
Batch 60 | Samples 6000-6100 | Loss = 237.5224
Epoch 2 complete.

===== Epoch 3/10 =====
Batch 0 | Samples 0- 100 | Loss = 317.2501
Batch 10 | Samples 1000-1100 | Loss = 112.9827
Batch 20 | Samples 2000-2100 | Loss = 147.4664
Batch 30 | Samples 3000-3100 | Loss = 138.3893
Batch 40 | Samples 4000-4100 | Loss = 282.1378
Batch 50 | Samples 5000-5100 | Loss = 388.8303
Batch 60 | Samples 6000-6100 | Loss = 299.2487
Epoch 3 complete.
```

Figure Note. *Console output illustrating the training progress of the neural network using mini-batch gradient descent. The model was trained for ten epochs with a batch size of 100*

compounds, and the loss was reported at regular intervals throughout each epoch. Although the loss fluctuates between individual mini-batches because each batch contains different molecular fingerprints, the overall reduction in loss across successive epochs indicates that the model is learning meaningful patterns from the training data and improving its predictive performance.

Training Methodology

After the neural network architecture was constructed, the model was trained using mini-batch gradient descent with the Adam optimization algorithm. Rather than processing the entire training dataset simultaneously, the model learned from mini-batches containing 100 chemical compounds. This approach reduces memory requirements while allowing the optimizer to update the network weights more frequently during each training epoch. The model was trained for ten epochs, meaning the complete training dataset was presented to the network ten separate times.

During each training iteration, the neural network generated toxicity predictions for the current mini-batch, compared those predictions with the known class labels using the binary cross-entropy loss function, and adjusted the model parameters through backpropagation. As training progressed, the objective was to minimize the loss function, indicating that the model's predictions were becoming more consistent with the observed toxicity labels. TensorBoard was used to record the loss after each optimization step, providing a visual representation of model convergence throughout the training process.

Figure 4. *Validation Predictions and Model Accuracy*

```
Generating predictions on validation dataset...
Predictions generated successfully.
Prediction shape: (782, 1)

Validation Accuracy
-----
Accuracy: 0.9092

Training complete.

Process finished with exit code 0

> assignments > CTA4-Toxicology-Classification > tox21_classifier.py
```

Figure Note. *Console output showing successful generation of predictions for the validation dataset following completion of model training. The neural network produced binary toxicity predictions for 782 validation compounds and achieved a validation accuracy of **90.92%**, indicating that the model correctly classified approximately nine out of every ten compounds in the validation set. This evaluation confirms that the trained neural network generalized effectively to previously unseen molecular data.*

Results

Following completion of training, the neural network generated predictions for the validation dataset containing 782 previously unseen chemical compounds. The prediction output consisted of a single probability for each compound, which was converted into a binary toxicity classification using the sigmoid output layer. Model performance was evaluated using classification accuracy, resulting in a validation accuracy of 90.92%.

This result indicates that the neural network correctly classified approximately nine out of every ten compounds within the validation dataset. The relatively high validation accuracy suggests that the model successfully learned meaningful relationships between the molecular fingerprint features and the associated toxicity labels while maintaining good generalization performance. Although accuracy provides a useful measure of overall performance, additional evaluation metrics such as precision, recall, F1-score, and the area under the receiver operating characteristic (ROC) curve (ROC-AUC) would provide a more comprehensive assessment of the classifier, particularly for datasets with potential class imbalance.

TensorBoard Visualization

TensorBoard was used throughout training to monitor model performance and verify that the neural network architecture was implemented as intended. The Time Series visualization recorded the binary cross-entropy loss after each mini-batch optimization step, providing a graphical representation of model convergence over time. Although the loss fluctuated between individual mini-batches because each batch contained different molecular compounds, the overall downward trend and smoothed loss curve demonstrate that the Adam optimizer successfully minimized the loss function as training progressed. In addition to monitoring convergence, TensorBoard generated a computation graph illustrating the complete neural network architecture. The graph confirms the flow of data through the model, beginning with the input placeholders, followed by the hidden layer, dropout regularization, output layer, loss calculation, and finally the optimization process. Together, these visualizations provide both quantitative and structural validation that the neural network was implemented correctly and learned meaningful patterns from the Tox21 molecular fingerprint data.

Figure 5. *TensorBoard Time Series Showing Model Convergence*

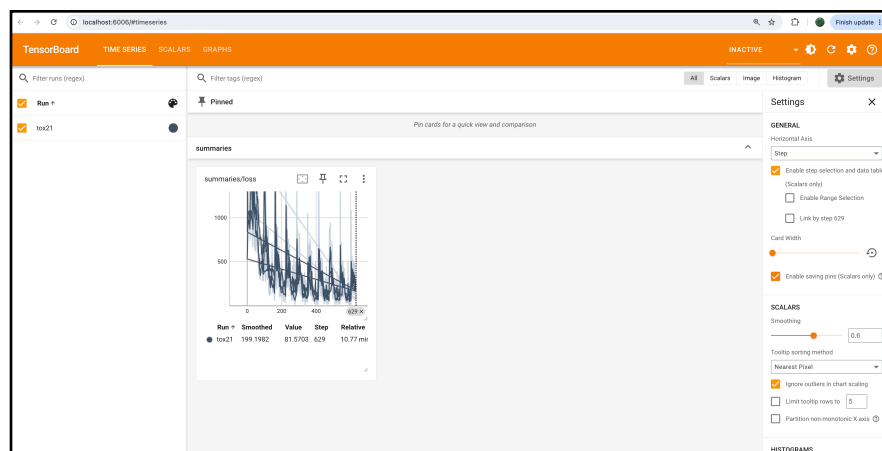


Figure Note. *TensorBoard time series visualization of the binary cross-entropy loss recorded during neural network training. Although individual mini-batch losses fluctuate because each optimization step is performed on a different subset of molecular compounds, the smoothed trend demonstrates a consistent reduction in loss throughout training. This convergence indicates that the model progressively learned predictive relationships within the Tox21 molecular fingerprint features.*

Figure 6. *TensorBoard Computation Graph of the Neural Network Architecture*

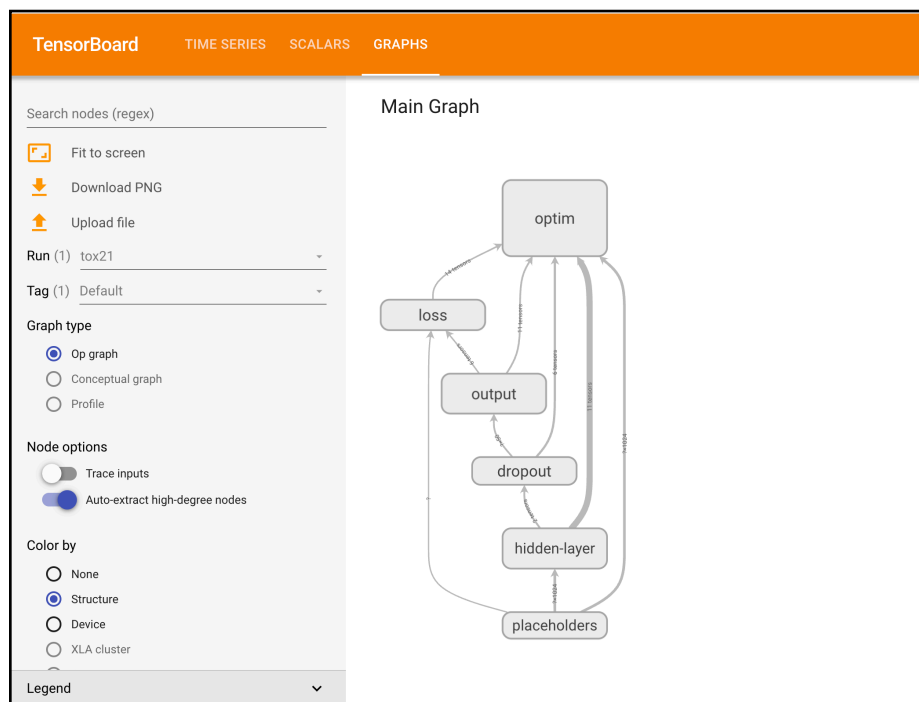


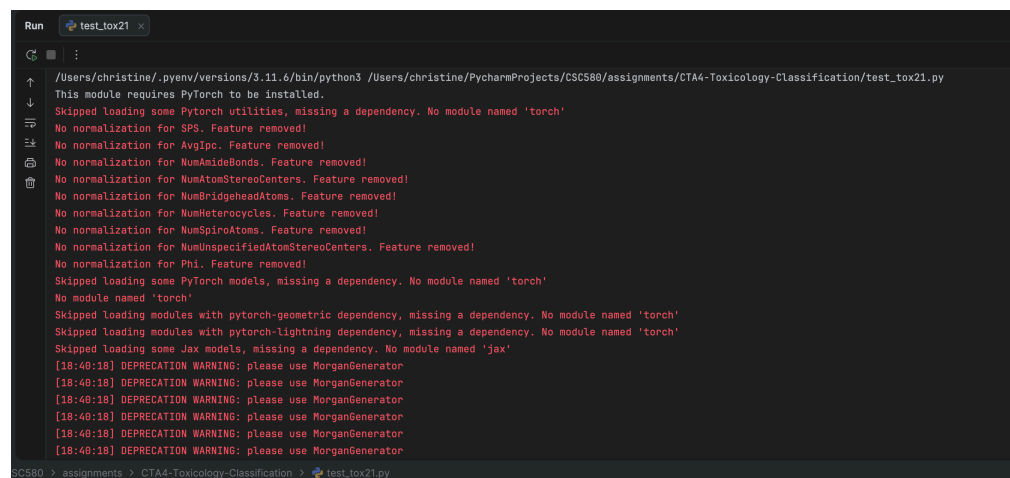
Figure Note. *TensorBoard computation graph illustrating the complete feedforward neural network architecture used for toxicology classification. The graph depicts the sequential flow of data through the input placeholders, hidden layer, dropout regularization, output layer, loss function, and Adam optimizer. This visualization confirms that the implemented model architecture matched the intended computational design and provides a graphical representation of the training pipeline.*

Challenges and Lessons Learned

One of the most significant challenges encountered during this project involved adapting legacy machine learning code to a modern scientific computing environment. The assignment was originally written for TensorFlow 1.x and an earlier DeepChem software stack, whereas the implementation was completed using TensorFlow 2.21, DeepChem 2.8.0, and Python 3.11. As a result, several compatibility issues required modifications before the neural network could be successfully trained. Legacy TensorFlow functions, including `tf.set_random_seed()`, `tf.placeholder()`, and `tf.train.AdamOptimizer()`, were replaced with their TensorFlow 2.x compatibility equivalents to preserve the original functionality while maintaining compatibility with the updated framework.

Additional challenges were encountered when loading the Tox21 dataset using DeepChem. Initially, the combination of DeepChem 2.8.0 and the most recent RDKit release generated repeated deprecation warnings related to the transition from the legacy Morgan fingerprint implementation to the newer MorganGenerator interface. Although these warnings initially appeared to indicate a loading failure, further investigation revealed that the primary issue was a compatibility mismatch between DeepChem and the latest RDKit release. Downgrading RDKit to a stable, supported version restored compatibility and allowed the Tox21 dataset to load successfully without interrupting the preprocessing pipeline. Figure A1 illustrates the compatibility issues encountered during the initial implementation. This experience reinforced the importance of dependency management, software version compatibility, and careful debugging when developing machine learning applications in rapidly evolving scientific software ecosystems.

Figure A1. *RDKit and DeepChem Compatibility Issues During Initial Dataset Loading*



```
Run test_tox21
/Users/christine/.pyenv/versions/3.11.6/bin/python3 /Users/christine/PycharmProjects/CSC580/assignments/CTA4-Toxicology-Classification/test_tox21.py
This module requires PyTorch to be installed.
Skipped loading some PyTorch utilities, missing a dependency. No module named 'torch'
No normalization for SPS. Feature removed!
No normalization for AvgIpc. Feature removed!
No normalization for NumAmideBonds. Feature removed!
No normalization for NumAtomStereoCenters. Feature removed!
No normalization for NumBridgeheadAtoms. Feature removed!
No normalization for NumHeterocycles. Feature removed!
No normalization for NumSpiralAtoms. Feature removed!
No normalization for NumUnspecifiedAtomStereoCenters. Feature removed!
No normalization for Phi. Feature removed!
Skipped loading some PyTorch models, missing a dependency. No module named 'torch'
No module named 'torch'
Skipped loading modules with pytorch-geometric dependency, missing a dependency. No module named 'torch'
Skipped loading modules with pytorch-lightning dependency, missing a dependency. No module named 'torch'
Skipped loading some Jax models, missing a dependency. No module named 'jax'
[18:40:18] DEPRECATION WARNING: please use MorganGenerator
[18:40:18] DEPRECATION WARNING: please use MorganGenerator
[18:40:18] DEPRECATION WARNING: please use MorganGenerator
[18:40:18] DEPRECATION WARNING: please use MorganGenerator
[18:40:18] DEPRECATION WARNING: please use MorganGenerator
[18:40:18] DEPRECATION WARNING: please use MorganGenerator
```

Figure Note. *Console output illustrating the deprecation warnings and compatibility issues encountered when loading the Tox21 dataset using the latest RDKit release with DeepChem*

2.8.0. The warnings referenced the transition from the legacy Morgan fingerprint implementation to the newer MorganGenerator interface and ultimately required the installation of a compatible RDKit version before the dataset could be successfully processed.

Conclusion

This project demonstrated the complete workflow required to develop, train, and evaluate a feedforward neural network for binary toxicology classification using the Tox21 dataset. From data preprocessing and neural network construction to mini-batch training, TensorBoard visualization, and model evaluation, each stage provided practical experience with the fundamental concepts underlying modern deep learning. The resulting model achieved a validation accuracy of 90.92%, demonstrating that the neural network successfully learned predictive patterns from molecular fingerprint data while maintaining good generalization performance on previously unseen compounds.

Beyond the successful implementation of the model, this project highlighted the importance of software compatibility, reproducibility, and model interpretability in applied machine learning. Resolving framework and library compatibility issues emphasized that developing machine learning systems often requires as much engineering effort as algorithm development. Although this assignment focused on toxicology prediction, the techniques explored throughout the project are broadly applicable to biomedical research and healthcare. Understanding how neural networks process complex biomedical data provides a strong foundation for future work involving clinical decision support, biomedical pattern recognition, and the responsible development of artificial intelligence systems that augment human expertise rather than replace it.

References

DeepChem Developers. (2024). *DeepChem: Democratizing deep learning for drug discovery, quantum chemistry, materials science and biology* (Version 2.8.0) <https://deepchem.io/>

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press. <https://www.deeplearningbook.org/>

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR)* <https://arxiv.org/abs/1412.6980>

Ramsundar, B., Eastman, P., Walters, P., Pande, V., Leswing, K., & Wu, Z. (2019). *Deep learning for the life sciences: Applying deep learning to genomics, microscopy, drug discovery, and more*. O'Reilly Media.

TensorFlow Developers. (2024). *TensorFlow documentation*. <https://www.tensorflow.org/>